Exception Handling in PL/SQL – Complete Notes

---

1. Introduction to Exception Handling

An **Exception** is an error that disrupts the normal flow of a program.

PL/SQL provides an **EXCEPTION block** to:

- Detect errors
- Handle errors
- Prevent program termination
- Provide meaningful error messages

---

2. Basic Structure of Exception Handling

Syntax

```
DECLARE
  -- Declaration statements
BEGIN
  -- SQL statements
  -- Procedural statements
EXCEPTION
  -- Exception handling statements
END;
```

---

3. Types of Exceptions in PL/SQL

There are two main types:

1. System-Defined Exceptions
2. User-Defined Exceptions

---

4. System-Defined Exceptions

These are **predefined exceptions** raised automatically by Oracle when rules are violated.

Examples:

- NO_DATA_FOUND
- ZERO_DIVIDE

- TOO_MANY_ROWS
- DUP_VAL_ON_INDEX
- LOGIN_DENIED

Example: ZERO_DIVIDE Exception

```
DECLARE
  a NUMBER := 10;
  b NUMBER := 0;
  c NUMBER;
BEGIN
  c := a / b;  -- Division by zero
  DBMS_OUTPUT.PUT_LINE('Result: ' || c);
EXCEPTION
  WHEN ZERO_DIVIDE THEN
    DBMS_OUTPUT.PUT_LINE('Error: Division by zero is not allowed.');
END;
```

Output:

Error: Division by zero is not allowed.

Explanation:

- a = 10, b = 0
- Division causes ZERO_DIVIDE error
- Exception block catches and handles it

---

5. Kinds of System-Defined Exceptions

System-defined exceptions are divided into:

1 Named System Exceptions
2 Unnamed System Exceptions

5.1 Named System Exceptions

These have predefined names.

Examples:

- ACCESS_INTO_NULL
- DUP_VAL_ON_INDEX
- LOGIN_DENIED

Example 1: ACCESS_INTO_NULL

```
DECLARE
  v_null_var VARCHAR2(10);
BEGIN
  BEGIN
    IF v_null_var = 'test' THEN
      NULL;
    END IF;
  EXCEPTION
    WHEN ACCESS_INTO_NULL THEN
      DBMS_OUTPUT.PUT_LINE('Caught ACCESS_INTO_NULL exception');
  END;
END;
```

Example 2: DUP_VAL_ON_INDEX

```
BEGIN
  INSERT INTO employees (employee_id, name)
  VALUES (1, 'John Doe');

  INSERT INTO employees (employee_id, name)
  VALUES (1, 'Jane Doe');

  COMMIT;
EXCEPTION
  WHEN DUP_VAL_ON_INDEX THEN
    DBMS_OUTPUT.PUT_LINE('Duplicate value detected');
END;
```

Example Table Creation (geeks)

```
CREATE TABLE geeks (
  g_id INT PRIMARY KEY,
  g_name VARCHAR2(20),
  marks INT
);

INSERT INTO geeks VALUES (1, 'Suraj', 100);
INSERT INTO geeks VALUES (2, 'Praveen', 97);
INSERT INTO geeks VALUES (3, 'Jessie', 99);
```

Primary key ensures unique g_id values.

5.2 Unnamed System Exceptions

These exceptions:

- Do not have predefined names
- Are identified using error codes
- Use PRAGMA EXCEPTION_INIT

Syntax

```
DECLARE
  exception_name EXCEPTION;
  PRAGMA EXCEPTION_INIT(exception_name, -error_code);
BEGIN
  -- Code
EXCEPTION
  WHEN exception_name THEN
    -- Handling code
END;
```

Example: Foreign Key Violation

```
DECLARE
  fk_violation EXCEPTION;
  PRAGMA EXCEPTION_INIT(fk_violation, -2292);
BEGIN
  DELETE FROM departments
  WHERE department_id = 10;
EXCEPTION
  WHEN fk_violation THEN
    DBMS_OUTPUT.PUT_LINE(
    'Error: Cannot delete department. Employees exist in it.');
END;
```

Explanation:

- -2292 = Foreign key constraint violation
- Exception is mapped using PRAGMA
- Proper error message is displayed

---

6. User-Defined Exceptions

These are custom exceptions created by the programmer.

Used for:

- Business rule validation
- Custom error handling
- Logical validation errors

Syntax

```
DECLARE
  exception_name EXCEPTION;
BEGIN
  IF condition THEN
    RAISE exception_name;
  END IF;
EXCEPTION
```

```
  WHEN exception_name THEN
    DBMS_OUTPUT.PUT_LINE('Exception handled');
END;
```

Example: Division Check Program

Conditions:

- y ≠ 0
- y ≤ x

```
DECLARE
  x INT := &x;
  y INT := &y;
  div_r FLOAT;
  exp1 EXCEPTION;  -- division by zero
  exp2 EXCEPTION;  -- y > x
BEGIN
  IF y = 0 THEN
    RAISE exp1;
  ELSIF y > x THEN
    RAISE exp2;
  ELSE
    div_r := x / y;
    DBMS_OUTPUT.PUT_LINE('The result is ' || div_r);
  END IF;
EXCEPTION
  WHEN exp1 THEN
    DBMS_OUTPUT.PUT_LINE('Error: Division by zero is not allowed');
  WHEN exp2 THEN
    DBMS_OUTPUT.PUT_LINE(
    'Error: y is greater than x, please check the input');
END;
```

---

7. RAISE_APPLICATION_ERROR

Used to create custom Oracle error messages.

Syntax

RAISE_APPLICATION_ERROR(error_number, 'error_message');

- Error number must be between -20000 and -20999

Example

```
DECLARE
  myex EXCEPTION;
  n NUMBER := 10;
BEGIN
  FOR i IN 1..n LOOP
    DBMS_OUTPUT.PUT_LINE(i*i);

    IF i*i = 36 THEN
      RAISE myex;
    END IF;
  END LOOP;
EXCEPTION
  WHEN myex THEN
    RAISE_APPLICATION_ERROR(-20015,
    'Welcome to GeeksForGeeks');
END;
```

Output:

```
1
4
9
16
25
36
ORA-20015: Welcome to GeeksForGeeks
```

---

8. Summary Table

Type	Defined By	Raised By	Example
System-Defined	Oracle	Automatically	ZERO_DIVIDE
Unnamed System	Oracle	Automatically	-2292
User-Defined	Programmer	RAISE	Business rule errors

---

9. Key Points for Exams & Interviews

- EXCEPTION block handles runtime errors.
- ZERO_DIVIDE occurs when dividing by zero.
- DUP_VAL_ON_INDEX occurs when duplicate primary key inserted.
- PRAGMA EXCEPTION_INIT maps error codes.
- RAISE is used for user-defined exceptions.
- RAISE_APPLICATION_ERROR creates custom Oracle errors.

Final Quick Revision

Exception = Error
System Exception = Oracle-defined
User Exception = Programmer-defined
RAISE = Trigger exception
PRAGMA = Map error code
RAISE_APPLICATION_ERROR = Custom error message